

CHRONIX EDU

Agent File

Stack Rules, Conventions & Boundaries for AI-Assisted Development

Version 1.0 | Chronix Technology Limited

PURPOSE

This document is the institutional memory of Chronix Edu. Every rule here exists because the alternative creates bugs, inconsistency, or rework. Claude must follow every rule in this file on every prompt. No exceptions. No improvisation.

Product	Chronix Edu — School Management System
Repository	github.com/chronixtech/chronixedu (single repo)
Structure	/apps/web /apps/api /packages/shared
Domain	edu.chronixtechnology.com
Primary AI Tool	Claude (claude.ai or API)
Code Review	None — straight to deployment
Agent File Version	1.0
Last Updated	June 2025

1. The Locked Stack

These are the only technologies permitted in Chronix Edu. Claude must not suggest, introduce, or use any library, framework, or service not on this list without explicit written approval. If a task cannot be solved with these tools, raise it as a decision — do not substitute silently.

Layer	Locked Tool	Do NOT substitute with
Frontend Framework	Next.js 14 (App Router)	React alone, Vite, Remix, Nuxt
Styling	Tailwind CSS	Bootstrap, Material UI, styled-components, Chakra
Backend	Node.js + Express (TypeScript)	Fastify, NestJS, Hono, plain Node http
Database	PostgreSQL via Supabase	MongoDB, Firebase Firestore, MySQL, SQLite
Auth	Supabase Auth + JWT	NextAuth alone, Clerk, Auth0, Passport.js
File Storage	Supabase Storage	AWS S3, Cloudinary, Firebase Storage
PDF	Puppeteer (headless Chrome)	pdfkit, jsPDF, React-PDF, wkhtmltopdf
Offline Storage	Dexie.js (IndexedDB)	localForage, localStorage, PouchDB
Email	SendGrid (@sendgrid/mail)	Nodemailer, Mailgun, Resend, AWS SES
SMS	Termii (Phase 3 only)	Twilio, Africa's Talking, Vonage
Payments	Paystack (Phase 3 only)	Stripe, Flutterwave, Squad
Notifications	Firebase Cloud Messaging	OneSignal, Pusher, raw WebSocket
Hosting — Web	Vercel	Netlify, AWS Amplify, self-hosted
Hosting — API	Railway	Heroku, Render, AWS EC2, DigitalOcean
Language	TypeScript (strict mode)	JavaScript (.js files), Flow
Package Manager	npm workspaces	yarn, pnpm, bun
Form Library	React Hook Form	Formik, react-final-form, uncontrolled forms
Charts (Phase 3)	Recharts	Chart.js, D3, Victory, ApexCharts
Drag and Drop	@dnd-kit/core (Phase 3)	react-beautiful-dnd, react-dnd

2. Repository & Folder Structure

NON-NEGOTIABLE

Every file Claude creates must go in exactly the location specified below. Creating files in the wrong location breaks imports, deployments, and the ability of other prompts to reference them.

2.1 Root Structure

```
chronixedu/
├── apps/
│   ├── web/
│   └── api/
├── packages/
│   └── shared/
├── .env.example
├── .gitignore
├── package.json
├── tsconfig.base.json
└── README.md
```

- ← GitHub repo root
- ← Next.js 14 frontend
- ← Express backend
- ← Shared TypeScript types only
- ← All required env vars documented
- ← Root workspace config
- ← Base TypeScript config

2.2 Frontend Structure — /apps/web

```
apps/web/
├── app/
│   ├── (auth)/
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   ├── principal/
│   │   └── teacher/
│   └── settings/
│   ├── (parent)/
│   ├── (student)/
│   └── (super-admin)/
├── components/
│   ├── ui/
│   └── [feature]/
├── hooks/
├── lib/
│   ├── api.ts
│   ├── auth.ts
│   └── utils.ts
├── services/
├── store/
├── types/
├── public/
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

- ← Next.js App Router pages
- ← Login, forgot password
- ← Admin, principal, teacher views
- ← Parent portal
- ← Student portal
- ← Chronix staff only
- ← Reusable UI (buttons, modals, tables)
- ← Feature-specific components
- ← Custom React hooks
- ← Axios instance with base URL + auth header
- ← Auth helpers
- ← Shared utilities
- ← API call functions (one file per module)
- ← Zustand global state
- ← Frontend-only types (re-exports from shared)

2.3 Backend Structure — /apps/api

```
apps/api/
├── src/
│   ├── middleware/
│   │   ├── auth.ts           ← verifyToken, requireRole
│   │   ├── validate.ts      ← Zod request validation
│   │   └── errorHandler.ts  ← Global error handler
│   ├── routes/              ← One file per module
│   │   ├── schools.ts
│   │   ├── students.ts
│   │   ├── scores.ts
│   │   ├── results.ts
│   │   ├── attendance.ts
│   │   └── ...
│   ├── services/            ← Business logic (no Express req/res)
│   │   ├── resultEngine.ts
│   │   ├── reportCard.ts
│   │   └── ...
│   ├── db/
│   │   ├── client.ts        ← Supabase client (anon + service role)
│   │   └── queries/         ← Raw SQL query functions
│   ├── templates/           ← Handlebars HTML templates (report cards)
│   ├── jobs/                ← Cron jobs (analytics, reminders)
│   ├── migrations/          ← SQL migration files
│   │   └── 001_initial_schema.sql
│   └── index.ts              ← Entry point
├── tsconfig.json
└── package.json
```

2.4 Shared Package — /packages/shared

```
packages/shared/
├── src/
│   ├── types/
│   │   ├── school.ts        ← School, SchoolSettings
│   │   ├── user.ts          ← User, UserRole, JWTClaims
│   │   ├── student.ts       ← Student, Parent, StudentClass
│   │   ├── academic.ts      ← Session, Term, Class, Subject
│   │   ├── scores.ts         ← Score, ResultStatus, AssessmentConfig
│   │   └── index.ts         ← Re-exports everything
│   └── constants/
│       ├── defaults.ts      ← Default grading scale, assessment weights
│       └── enums.ts         ← UserRole, ResultStatus, AttendanceStatus
├── tsconfig.json
└── package.json
```

3. Naming Conventions

Consistent naming is how a codebase stays readable when Claude generates code across 47 separate prompts. Every name below is a rule, not a suggestion.

3.1 Files and Folders

Type	Convention	Example
React components	<code>PascalCase.tsx</code>	<code>ScoreEntryGrid.tsx</code>
Pages (Next.js)	<code>lowercase / kebab-case</code>	<code>app/(dashboard)/teacher/scores/page.tsx</code>
API route files	<code>camelCase.ts</code>	<code>scores.ts, resultStatus.ts</code>
Service files	<code>camelCase.ts</code>	<code>resultEngine.ts, reportCard.ts</code>
Hook files	<code>use prefix + camelCase</code>	<code>useCurrentTerm.ts, useSchoolSettings.ts</code>
Utility files	<code>camelCase.ts</code>	<code>formatCurrency.ts, dateHelpers.ts</code>
Type files	<code>camelCase.ts</code>	<code>student.ts, scores.ts</code>
Migration files	<code>NNN_snake_case.sql</code>	<code>001_initial_schema.sql, 002_attendance.sql</code>
Folders	<code>lowercase kebab-case</code>	<code>score-entry/, result-approval/</code>

3.2 TypeScript & Variables

RULE N1 Use camelCase for variables and functions

RULE	All variables, function names, and object properties use camelCase. No snake_case in TypeScript (snake_case is for SQL only).
✗ WRONG	<code>const student_id = req.params.student_id</code>
✓ CORRECT	<code>const studentId = req.params.studentId</code>

RULE N2 Use PascalCase for types, interfaces, and enums

RULE	All TypeScript types, interfaces, and enums use PascalCase.
✗ WRONG	<code>type userRole = 'teacher' 'principal'</code>
✓ CORRECT	<code>type UserRole = 'teacher' 'principal'</code>

RULE N3 Database columns use snake_case in SQL only

RULE	SQL queries use snake_case column names. When mapping to TypeScript, convert to camelCase immediately at the query boundary.
------	--

✗ WRONG	<pre>// Using school_id directly in TypeScript logic const school_id = user.school_id</pre>
✓ CORRECT	<pre>// Map at DB boundary, use camelCase everywhere else const { school_id } = row const schoolId = school_id</pre>

3.3 API Routes

RULE N4 API routes follow REST conventions	
RULE	All API routes use lowercase kebab-case. No camelCase in URLs. Resources are plural nouns. Actions that are not CRUD use verb-noun format.
✗ WRONG	<code>POST /api/schools/:schoolId/generateReportCards</code>
✓ CORRECT	<code>POST /api/schools/:schoolId/report-cards/generate</code>

Method	Pattern	Example
GET	List	<code>GET /api/schools/:schoolId/students</code>
GET	Single	<code>GET /api/schools/:schoolId/students/:studentId</code>
POST	Create	<code>POST /api/schools/:schoolId/students</code>
PATCH	Update	<code>PATCH /api/schools/:schoolId/students/:studentId</code>
DELETE	Delete	<code>DELETE /api/schools/:schoolId/students/:studentId</code>
POST	Action	<code>POST /api/schools/:schoolId/results/publish</code>

4. Code Rules

These rules apply to every file Claude generates. No exceptions.

4.1 TypeScript Rules

RULE C1 Strict TypeScript — no 'any'

RULE	TypeScript strict mode is enabled. The type 'any' is banned. If a type is unknown, use 'unknown' and narrow it. Every function parameter and return value must be typed.
✗ WRONG	<pre>function getStudent(id: any): any { ... }</pre>
✓ CORRECT	<pre>function getStudent(id: string): Promise<Student> { ... }</pre>

RULE C2 Import with path aliases only — no relative paths going up

RULE	In both /apps/web and /apps/api, always use the @/ path alias. Never use ../../.. relative imports that go up more than one level.
✗ WRONG	<pre>import { verifyToken } from '../../../middleware/auth'</pre>
✓ CORRECT	<pre>import { verifyToken } from '@middleware/auth'</pre>

RULE C3 Shared types import from @chronixedu/shared

RULE	Types used by both frontend and backend must be imported from the shared package, never redefined locally.
✗ WRONG	<pre>// In apps/api/src/routes/students.ts type Student = { id: string; name: string; ... }</pre>
✓ CORRECT	<pre>import type { Student } from '@chronixedu/shared'</pre>

RULE C4 No console.log in production code

RULE	Use the Winston logger in the API. Use nothing (silent) in the frontend. console.log is for debugging only and must be removed before a prompt is considered complete.
✗ WRONG	<pre>console.log('Score saved:', score)</pre>
✓ CORRECT	<pre>logger.info('Score saved', { scoreId: score.id, userId: req.user.userId })</pre>

4.2 API Rules

RULE C5 Every route must have verifyToken and requireRole

RULE	No route is accessible without authentication. Every route file must apply <code>verifyToken</code> . Every route that has role restrictions must apply <code>requireRole</code> . No exceptions.
✗ WRONG	<pre>router.get('/students', async (req, res) => { ... })</pre>
✓ CORRECT	<pre>router.get('/students', verifyToken, requireRole('principal','registrar'), async (req, res) => { ... })</pre>

RULE C6 All API responses use the standard response envelope

RULE	Every API response — success or error — uses the same envelope format. No raw data responses. No raw error strings.
✗ WRONG	<pre>res.json(students) // raw array res.status(400).json('Bad request') // raw string</pre>
✓ CORRECT	<pre>res.json({ success: true, data: students }) res.status(400).json({ success: false, error: { code: 'VALIDATION_ERROR', message: 'school_id is required' } })</pre>

RULE C7 All inputs validated with Zod before processing

RULE	Every POST and PATCH route validates its request body with a Zod schema before any database operation. Invalid input returns 400 with the Zod error details.
✗ WRONG	<pre>const { name, email } = req.body // unvalidated</pre>
✓ CORRECT	<pre>const schema = z.object({ name: z.string().min(1), email: z.string().email() }) const result = schema.safeParse(req.body) if (!result.success) return res.status(400).json({ success: false, error: result.error })</pre>

RULE C8 All async route handlers wrapped in try/catch

RULE	Every async Express route handler must have a try/catch block that passes errors to the global error handler. Never let an unhandled promise rejection crash the server.
✗ WRONG	<pre>router.get('/students', verifyToken, async (req, res) => { const students = await db.query(...) res.json(students) })</pre>
✓ CORRECT	<pre>router.get('/students', verifyToken, async (req, res, next) => { try { const students = await db.query(...) res.json({ success: true, data: students }) } catch (err) { next(err) } })</pre>

4.3 Database Rules

RULE C9 Every query that writes data must include school_id from JWT

RULE	When inserting or updating records, always take school_id from req.user.schoolId (JWT claim), never from req.body or req.params. A user must never be able to write to another school's data.
✗ WRONG	<pre>const { school_id } = req.body // NEVER trust client-supplied school_id</pre>
✓ CORRECT	<pre>const schoolId = req.user.schoolId // Always from verified JWT claim</pre>

RULE C10 Use parameterised queries — never string interpolation in SQL

RULE	All SQL queries must use parameterised placeholders. Never concatenate user input into a SQL string. This prevents SQL injection.
✗ WRONG	<pre>const q = `SELECT * FROM students WHERE id = '\${studentId}'`</pre>
✓ CORRECT	<pre>const q = 'SELECT * FROM students WHERE id = \$1' await db.query(q, [studentId])</pre>

RULE C11 Audit log on every sensitive write

RULE	Score entry, result status changes, settings changes, and payment records must write to audit_logs immediately after the main write succeeds. Use the logAudit() helper function.
✗ WRONG	<pre>await db.query('UPDATE scores SET score = \$1 WHERE id = \$2', [score, id]) // No audit log</pre>
✓ CORRECT	<pre>await db.query('UPDATE scores SET score = \$1 WHERE id = \$2', [newScore, id]) await logAudit({ schoolId, userId, action: 'SCORE_UPDATE', entity: 'scores', entityId: id, oldValue: { score: oldScore }, newValue: { score: newScore } })</pre>

4.4 Frontend Rules

RULE C12 All API calls go through /lib/api.ts — never raw fetch

RULE	The frontend has a single Axios instance in /lib/api.ts that attaches the auth token automatically. Never use raw fetch() or create a second Axios instance.
✗ WRONG	<pre>const res = await fetch('/api/schools/' + schoolId + '/students', { headers: { Authorization: 'Bearer ' + token } })</pre>
✓ CORRECT	<pre>import api from '@lib/api' const res = await api.get(`/schools/\${schoolId}/students`)</pre>

RULE C13 No API calls directly in React components

RULE	API calls belong in /services/ files, not inside components. Components call service functions. This keeps components testable and API logic reusable.
✗ WRONG	<pre>// Inside ScoreEntry.tsx const res = await</pre>

	<pre>api.post('/scores/bulk-entry', scores)</pre>
✓ CORRECT	<pre>// In /services/scores.ts export const bulkEnterScores = (data) => api.post('/scores/bulk-entry', data) // In ScoreEntry.tsx await bulkEnterScores(scores)</pre>

RULE C14 All forms use React Hook Form — no uncontrolled inputs	
RULE	Every form in the application uses React Hook Form. No useState-managed form fields. No uncontrolled inputs. Validation via Zod resolver.
✗ WRONG	<pre>const [name, setName] = useState('') <input value={name} onChange={e => setName(e.target.value)} /></pre>
✓ CORRECT	<pre>const { register, handleSubmit } = useForm<FormData>({ resolver: zodResolver(schema) }) <input {...register('name')} /></pre>

5. Environment Variables

All environment variables must exist in `.env.example` with a description comment. Never hardcode a secret, API key, URL, or credential in code. The application must fail loudly on startup if a required variable is missing.

```
# — Supabase —
DATABASE_URL=                # PostgreSQL connection string
SUPABASE_URL=                # https://[project].supabase.co
SUPABASE_ANON_KEY=           # Public anon key (safe for frontend)
SUPABASE_SERVICE_ROLE_KEY=   # NEVER expose to frontend or client

# — Auth —
JWT_SECRET=                  # Min 32 chars, random string
NEXTAUTH_SECRET=             # Next.js auth secret
NEXTAUTH_URL=                # https://edu.chronixtechnology.com

# — Email —
SENDGRID_API_KEY=            # SendGrid API key
SENDGRID_FROM_EMAIL=         # noreply@chronixtechnology.com

# — Storage —
SUPABASE_STORAGE_BUCKET=     # school-assets

# — App —
NEXT_PUBLIC_API_URL=          # https://api.edu.chronixtechnology.com
NODE_ENV=                    # development | production
PORT=                         # API server port (default 3001)

# — Phase 3 (add when building Phase 3) —
# PAYSTACK_SECRET_KEY=
# TERMII_API_KEY=             # Per-school key, stored in DB not env
# FIREBASE_SERVER_KEY=
```

RULE E1 SUPABASE_SERVICE_ROLE_KEY is server-only — never in frontend

RULE	The service role key bypasses all Row Level Security. It must only be used in <code>/apps/api</code> . It must never appear in <code>/apps/web</code> or in any file that could be bundled and sent to a browser.
✗ WRONG	<code>// In apps/web/lib/supabase.ts createClient(url, process.env.SUPABASE_SERVICE_ROLE_KEY)</code>
✓ CORRECT	<code>// Service role key is ONLY used in apps/api/src/db/client.ts // Frontend always uses SUPABASE_ANON_KEY</code>

RULE E2 Startup validation — fail loudly if env vars missing

RULE	The API server must check all required environment variables on startup and throw a clear error listing missing variables. Silent failures from missing env vars cause mysterious bugs.
------	---

✗ WRONG`// No startup check — app starts and fails mysteriously later`**✓ CORRECT**

```
const required =
  ['DATABASE_URL', 'JWT_SECRET', 'SUPABASE_URL', 'SUPABASE_SERVICE_ROLE_KEY']
const missing = required.filter(k => !process.env[k])
if (missing.length) throw new Error(`Missing env vars: ${missing.join(', ')}`)
```

6. Security Rules

Security is not a phase 3 concern. Every rule below applies from the first line of code written.

RULE S1 school_id always comes from JWT — never from request body

RULE	When scoping any query to a school, use req.user.schoolId from the verified JWT. Never accept school_id from req.body, req.params, or req.query for data write operations.
✗ WRONG	<pre>const schoolId = req.body.schoolId // Client can fake this</pre>
✓ CORRECT	<pre>const schoolId = req.user.schoolId // Verified by JWT middleware</pre>

RULE S2 Parent data access only through parent_students junction

RULE	When a parent requests student data, the query must JOIN through the parent_students table. Never accept a studentId from a parent request without verifying the parent-student relationship.
✗ WRONG	<pre>// Just fetches by studentId — any parent could access any student SELECT * FROM students WHERE id = \$1</pre>
✓ CORRECT	<pre>// Verifies parent owns this student before returning data SELECT s.* FROM students s JOIN parent_students ps ON ps.student_id = s.id WHERE s.id = \$1 AND ps.parent_id = \$2</pre>

RULE S3 Result status transitions are one-directional

RULE	Result status can only move forward: draft → submitted → approved → published. The only backward transition is via the explicit /results/return endpoint (principal only). The validateStatusTransition() function must be called before every status update.
✗ WRONG	<pre>// Just updates status without checking current state await db.query('UPDATE result_status SET status = \$1', ['published'])</pre>
✓ CORRECT	<pre>const valid = validateStatusTransition(currentStatus, 'published') if (!valid) return res.status(400).json({ success: false, error: { code: 'INVALID_TRANSITION' } })</pre>

RULE S4 Settings locking — 423 before any config change with live data

RULE	Assessment config changes return 423 Locked if scores exist for the term. Grading scale changes return 200 with a warnings array if published results exist. Session activation requires confirm:true in body.
✗ WRONG	<pre>// Just saves the new config without checking current state await db.query('UPDATE assessment_configs SET ...')</pre>
✓ CORRECT	<pre>const scoresExist = await checkScoresExistForTerm(configId, termId) if (scoresExist) return res.status(423).json({ success: false, error: { code: 'CONFIG_LOCKED', message: 'Scores entered for this term.' } })</pre>

RULE S5 Rate limiting on all API routes	
RULE	express-rate-limit is applied globally: 100 requests per minute per IP for general routes, 5 per minute for authentication routes. Applied in index.ts before any routes are registered.
✗ WRONG	// No rate limiting – open to brute force and DoS
✓ CORRECT	import rateLimit from 'express-rate-limit' app.use('/api/auth', rateLimit({ windowMs: 60000, max: 5 }))) app.use('/api', rateLimit({ windowMs: 60000, max: 100 })))

7. What Claude Must Never Do

These are hard boundaries. If a prompt asks Claude to do any of the following, Claude must refuse and explain why, then offer the correct alternative.

Never Do This	Why
Use a library not in the locked stack	Introduces untested dependencies and breaks the agent file contract
Use MongoDB, Firebase Firestore, or MySQL as the primary database	PostgreSQL via Supabase is locked. Switching databases mid-build is catastrophic.
Expose SUPABASE_SERVICE_ROLE_KEY to the frontend	Bypasses all RLS — every school's data becomes accessible to everyone
Skip verifyToken or requireRole on any API route	Creates an open endpoint — any person with the URL can access school data
Accept school_id from req.body for data writes	Client-controlled school_id allows cross-tenant data manipulation
Use raw fetch() in the frontend instead of /lib/api.ts	Bypasses auth token injection and response normalisation
Use useState to manage form state	Forms use React Hook Form only — this rule exists to prevent inconsistency across 47 prompts
Write console.log in production code	Use Winston logger in the API — console.log is stripped in production
Store data in localStorage or sessionStorage	Not supported in the deployment environment — use Dexie.js for offline, Zustand for state
Create files outside the defined folder structure	Breaks imports, deployment configs, and the ability of other prompts to reference files
Skip the Zod validation on any POST or PATCH route	Unvalidated inputs reach the database — source of data corruption bugs
Hardcode school-specific data (logo, name, colours) in code	All school-specific data comes from school_settings — hardcoding breaks multi-tenancy
Build Phase 3 or Phase 4 features before Phase 2 is complete	Out-of-scope work consumes timeline without delivering the launch goal
Suggest architectural changes without flagging them explicitly	Changes to the data model, folder structure, or stack must be flagged as DECISION REQUIRED — not silently implemented

8. Seeded Default Values

These values are seeded into the database when a new school is onboarded. They are the Nigerian standard defaults. Every school can change them through the admin settings panel — but this is what they start with.

8.1 Default Grading Scale

Grade	Min	Max	Label	Remark
A	70	100	Excellent	Outstanding performance
B	60	69	Very Good	Above average performance
C	50	59	Good	Average performance
D	40	49	Pass	Below average but passing
F	0	39	Fail	Below passing mark

PROMOTION CUTOFF	Default promotion cutoff is 40 (grade D). A student with an overall average below 40 is flagged as at-risk. Schools can change this in the Grading Scale settings page.
------------------	---

8.2 Default Assessment Components

Component	Max Score	Weight %	Display Order
CA 1 (First Continuous Assessment)	10	10%	1
CA 2 (Second Continuous Assessment)	10	10%	2
Mid-Term Test	10	10%	3
Examination	70	70%	4

TOTAL = 100%	CA 1 (10%) + CA 2 (10%) + Mid-Term (10%) + Exam (70%) = 100%. The system will reject any assessment config where weights do not sum to exactly 100%.
--------------	--

8.3 Default Academic Calendar

Term	Typical Start	Typical End	Note
First Term	September	December	Launch term — Child Prime Onyx goes live here
Second Term	January	April	
Third Term	May	July	

9. Chronix Technology Branding

When Chronix Edu branding appears in the product (super admin screens, login page, email templates), use these exact values.

Token	Value	Usage
Primary (Dark Blue)	#003366	Main navigation, headings, primary buttons
Accent (Orange)	#FF761B	CTAs, highlights, badges, active states
Secondary (Light Blue)	#2472B4	Links, secondary buttons, info states
Background	FFFFFF	Page background
Text	#333333	Body text, labels
Heading Font	Poppins (600 semi-bold)	All headings, product name, navigation labels
Body Font	Inter (400 regular)	All UI text, form labels, table content, paragraphs
Product Name	Chronix Edu	Always two words, capital C, capital E
Company Name	Chronix Technology Limited	Full legal name in contracts and footers
Slogan	Innovating Beyond Time	Footer and about page only
Domain	edu.chronixtechnology.com	All links and email from addresses

IMPORTANT

Each school's portal uses their own branding — their logo, name, and colours from school_settings.identity_config. Chronix branding appears only on the login page, super admin screens, and system emails. Never mix school branding with Chronix branding.